INNOPOLIS UNIVERSITY

Bachelor's Thesis

Initial draft — 13 October 2026

Performance Optimization of Large Language Model Inference on GPUs

Bulat Sharipov

Supervisor: [to be confirmed]

Innopolis, 2027

Contents

Abstract **6**

1 Introduction **7**

1.1 Motivation 7

1.2 Research problem and scope 8

1.3 Research questions 8

1.4 Contributions and current status 9

2 Literature Review **10**

2.1 Autoregressive inference 10

2.2 Serving engines and scheduling 11

2.3 Speculative decoding 12

2.4 Quantization 13

2.5 Attention kernels and GPU execution 13

2.6 Research gap addressed by this thesis 14

3 Methodology **15**

3.1 Study design 15

3.2 Model and recorded environment 16

3.3 Workload construction 17

3.4 Load protocol 18
3.5 Metric definitions 19
3.6 Optimization and comparison protocol 20
3.7 Quality and validity controls 21

4 Implementation 22
4.1 Repository organization 22
4.2 Prompt preparation and load generation 23
4.3 Baseline launcher and provenance 24
4.4 Prepared DFlash integration 24
4.5 Remaining integration work 25

5 Evaluation and Discussion 26
5.1 Available evidence 26
5.2 Performance at concurrency one 27
5.3 Behavior as offered load increases 28
5.4 Planned optimization comparisons 30
5.5 Current answers to the research questions 31

6 Conclusion 32

References 33

A Complete Baseline Results 36

B Evidence and Completion Checklist 41
B.1 Traceability 41
B.2 Outstanding evidence 42

List of Tables

3.1 Recorded baseline environment 16
3.2 Prepared input lengths, including the chat template 18

5.1 Baseline at concurrency one 27
5.2 Baseline at 16 offered requests/s 29
5.3 Experiments required to complete the evaluation 31

A.1 Baseline request outcomes. Error counts are zero for all runs. . . 36
A.2 Complete baseline: approximately 100 requested output tokens . 38
A.3 Complete baseline: approximately 500 requested output tokens . 39
A.4 Complete baseline: approximately 1000 requested output tokens 40

List of Figures

5.1 Baseline throughput versus offered load 28
5.2 Tail latency and completion for long responses 30

Abstract

Large language model inference involves interacting computational, memory and scheduling costs. This thesis investigates a systematic pipeline for improving GPU inference through serving-engine configuration, speculative decoding, quantization, optimized kernels and scheduling changes. The objective is to identify combinations that improve latency and throughput while preserving an explicitly defined level of output quality.

The implemented experimental foundation uses Qwen3-8B in BF16 on one NVIDIA H100 80 GB GPU, served by vLLM and measured with AIPerf. A reproducible workload contains 100 Russian and 100 English WildChat histories, short inputs, and three instruction-based output-length targets. The first baseline series comprises 36 load conditions. Across that series, 22,459 of 22,843 submitted requests completed; 384 were cancelled after the benchmark grace period. The results establish reference measurements and expose output-length variation and high-load degradation that subsequent comparisons must control.

This initial draft documents the methodology, implemented benchmark and available baseline results. Optimized configurations, quality evaluation, repeated trials and final conclusions remain pending. Their locations and required evidence are marked explicitly throughout the text.

Chapter 1

Introduction

1.1 Motivation

Large language models (LLMs) support conversational assistants, coding tools and applications that make repeated generation calls. Their usefulness depends on response quality and on how quickly a serving system can process requests. Sequential token generation makes long responses expensive in time, while growing demand requires the same GPU resources to serve many users. Efficient inference therefore requires attention to both individual response latency and aggregate throughput [1], [2].

An inference request has two principal phases. During *prefill*, the model processes the input and constructs attention states. During *decode*, it generates output tokens while reusing those states. A deployment must also allocate memory, schedule requests and transfer responses to clients. Improving one component can expose a different bottleneck, and increasing batch size can improve aggregate throughput while increasing the time experienced by each user [3].

Several optimization families address this problem. Serving engines manage batches and attention caches; quantization reduces numerical precision; speculative decoding verifies multiple proposed tokens together; and specialized GPU kernels improve execution of individual operations. The practical question is how these techniques interact in a complete deployment.

1.2 Research problem and scope

This thesis studies a profiling-guided optimization pipeline: establish a reproducible baseline, identify bottlenecks, evaluate individual techniques, combine compatible changes and remeasure the whole system. The word *pipeline* refers here to this sequence of optimization and evaluation stages. It does not imply that every technique must be enabled or that model layers are distributed through pipeline parallelism.

The initial experimental scope is deliberately concrete: Qwen3-8B, one H100 GPU, Russian and English requests, and short inputs. The repository currently contains a complete baseline series and scripts for a DFlash deployment [4], [5]. Comparisons with SGLang, quantized models, alternative drafters and custom kernels are planned extensions. Claims about other model sizes, long contexts or GPU architectures will require additional experiments.

1.3 Research questions

RQ1. What latency and throughput gains can a coordinated optimization pipeline achieve over default and tuned serving baselines, subject to a defined output-quality constraint?

RQ2. Which optimization combinations produce complementary gains, diminishing returns or regressions, and what measured bottlenecks explain these interactions?

RQ3. How does the preferred configuration change with language, output length and offered load, and can workload-based selection improve performance on held-out requests?

1.4 Contributions and current status

The intended contribution is a reproducible empirical method for selecting and combining inference optimizations under a common workload and evaluation protocol. Existing serving systems already integrate multiple techniques, so integration alone is insufficient as a novelty claim [1], [6], [7]. The research contribution will depend on explaining interactions, documenting unsuccessful combinations and establishing useful selection rules.

The work completed at this stage comprises a versioned workload-construction procedure, a benchmark runner, a recorded deployment configuration and 36 baseline reports. The draft adds an organized analysis of those reports. No end-to-end optimization speedup or quality-preservation result has yet been established in the repository snapshot reviewed here.

[Pending: E1–E6: complete the optimization experiments and replace the intended contributions with findings supported by measurements.]

Chapter 2

Literature Review

2.1 Autoregressive inference

An autoregressive language model generates a sequence by conditioning each token on the prompt and preceding outputs. For prompt x and output $y_{1:m}$,

$$p(y_{1:m} \mid x) = \prod_{t=1}^{m} p(y_t \mid x, y_{<t}). \tag{2.1}$$

This dependency limits straightforward parallel generation. A key–value (KV) cache stores attention keys and values so that previous token states can be reused. Its size grows with sequence length and the number of active requests. The resulting performance depends on model operations, memory traffic and the serving engine's ability to batch work [1], [8].

Prefill exposes parallel work across input tokens, whereas conventional decoding advances each active sequence one token at a time. These phases therefore need separate attention when interpreting measurements. The present benchmark

emphasizes decoding through short prompts and progressively longer requested responses; it does not characterize long-context prefill performance.

2.2 Serving engines and scheduling

vLLM introduced PagedAttention, which manages the KV cache in blocks to reduce fragmentation and redundant storage [1]. Better memory management permits more requests to be served together. However, an engine comparison must specify batch limits, model precision, cache policy and workload: a throughput figure is not an intrinsic property of the engine name.

SGLang combines a language for structured LLM programs with an execution runtime. Its RadixAttention mechanism reuses cached prefixes across requests [6]. This is relevant to applications with shared prompts and branching generation. The present baseline disables prefix caching, which limits the influence of repeated prompts but also excludes an important optimization opportunity. A future cache-enabled experiment must define prefix reuse explicitly.

Sarathi-Serve studies the interaction between prompt processing and decoding through chunked prefill and scheduling [3]. Its results motivate evaluating throughput together with latency rather than selecting a configuration solely by tokens per second. For this thesis, engine tuning includes batching limits and compatible execution settings, followed by measurements at the same offered loads. SGLang and scheduler modifications remain unmeasured extensions.

2.3 Speculative decoding

Speculative decoding uses a cheaper draft mechanism to propose several tokens and the target model to verify them together. With an appropriate acceptance and correction rule, exact speculative sampling preserves the target sampling distribution [8]. For a proposed token drawn from draft distribution q, the acceptance probability is $\min(1, p/q)$, with rejected proposals corrected using the positive residual between target and draft probabilities. This property concerns the specified target distribution; changing the target through quantization requires a separate quality assessment.

Acceleration depends on draft cost, verification cost and the number of useful tokens accepted per verification step. A high acceptance rate alone does not establish a speedup. At greater serving load, verification can consume execution capacity that would otherwise serve other requests.

EAGLE-3 replaces feature prediction with direct token prediction and combines features from multiple target-model layers. Its training-time test mechanism supports learning under conditions closer to inference [9]. It provides a relevant comparison for learned autoregressive drafting.

DFlash instead uses a lightweight block-diffusion model to draft tokens in parallel, conditioned on target-model context features [2]. This reduces sequential work within the drafter. A Qwen3-8B-compatible DFlash checkpoint is available, and the thesis repository includes a launcher using 15 speculative tokens [10], [11]. The existence of that launcher does not constitute a completed performance evaluation.

DSpark combines parallel drafting with a lightweight sequential component and adapts verification length using confidence and serving-load information [12].

It motivates studying verification efficiency under concurrency. A compatible implementation and checkpoint for the thesis setup must be confirmed before treating it as an experimental baseline.

2.4 Quantization

Quantization represents model parameters or activations at lower precision. It can reduce memory requirements and enable faster supported operations, but the result depends on the available kernels and the cost of scaling and conversion. AWQ uses activation information to guide low-bit weight quantization, while SmoothQuant redistributes activation outliers to enable 8-bit weight-and-activation computation [13], [14].

The planned study will evaluate supported FP8 and low-bit alternatives to BF16. Each candidate must be specified by the tensors quantized, calibration procedure and execution backend. Lower storage cost will not be treated as evidence of lower latency. Quantization also changes numerical outputs and potentially draft–target agreement, making its interaction with speculative decoding an experimental question.

2.5 Attention kernels and GPU execution

FlashAttention-3 targets Hopper GPUs using asynchronous execution and specialized kernel pipelines [15]. FlashAttention-4 studies different bottlenecks on Blackwell hardware [16]; it is relevant background, but Blackwell-only results cannot establish performance on the H100 used here. FlashInfer provides attention

implementations for serving workloads with different cache layouts and dynamic sequence shapes [7].

Kernel selection should follow profiling. Faster attention may produce a small end-to-end improvement when another operation or CPU scheduling dominates. Conversely, a custom fused kernel can be useful when it removes repeated memory transfers or launch overhead from a frequently executed path. The thesis will compare kernel time and complete-request performance wherever custom changes are introduced.

2.6 Research gap addressed by this thesis

The literature provides strong techniques at multiple levels of the inference stack. It also demonstrates that hardware, workload and scheduling conditions affect reported gains. This thesis addresses the practical problem of composing those techniques under one reproducible protocol. The central evidence will be individual, cumulative and component-removal comparisons, coupled with quality and completion measurements. Until those comparisons are completed, the contribution is an experimental framework and baseline characterization rather than a demonstrated universal acceleration method.

Chapter 3

Methodology

3.1 Study design

The unit of comparison is a fixed *deployment*: target checkpoint, numerical precision, serving engine, draft checkpoint if present, and server settings. Each deployment is evaluated over the same workload grid. Results from different deployments must retain separate identifiers so that configuration changes cannot be mistaken for workload effects [11].

The implemented baseline is denoted B_0. A tuned baseline B_1 will preserve the target model and precision while searching serving settings. Subsequent stages will evaluate speculative decoding, quantization and targeted kernel or scheduling changes. The final pipeline will be compared with both B_0 and B_1, so that gains from ordinary parameter tuning can be distinguished from gains due to additional methods.

3.2 Model and recorded environment

The target is the dense Qwen3-8B model [17], [18]. The recorded local configuration contains 36 layers, hidden dimension 4096, 32 attention query heads, 8 KV heads and head dimension 128. These values are reported from the experiment manifest rather than inferred from a generic model family description [5].

TABLE 3.1
Recorded baseline environment

Item	Recorded value
GPU	1 NVIDIA H100, 80 GB HBM3
Model and precision	Qwen3-8B, BF16; no drafter
Runtime	vLLM 0.30.0; AIPerf 0.13.0
Supporting packages	PyTorch 2.13.0; Transformers 5.17.0
Execution	CUDA graphs enabled; prefix caching disabled
Generation	Thinking disabled; temperature 0.6
Sampling	Top-p 0.95; top-k 20; model EOS defaults

The manifest identifies GPU 2 on a multi-GPU host. Other GPUs were occupied at startup, but shared CPU and memory contention were not measured. The record does not establish exclusive use of the entire host. Runtime package versions in Table 3.1 are distinct from version strings embedded in the downloaded model configuration.

[Pending: P1: record CPU and host-memory specifications, driver and CUDA versions, GPU power/clock settings, model and tokenizer revisions, and the resolved server configuration for future series.]

3.3 Workload construction

Requests are selected from WildChat-1M, a dataset of human–ChatGPT conversations [19]. The preparation script pins dataset revision 7d6490e462285cf85d91eabea0f9a954fbddcd1f and reads its first Parquet shard. It scans 5,316 rows and selects the first 100 distinct eligible histories in Russian and 100 in English [4], [11].

Each history ends at its last user message; later dataset responses are excluded. Conversation and last-user language labels must agree, message roles must be supported, and empty messages are rejected. Each selected history is tested with all three language-specific system instructions. The complete formatted input, including the chat template, must remain between 70 and 130 tokens in every profile. Thus the six request files contain 200 underlying histories reused across three output-length instructions, not 600 independent histories.

Thinking is disabled through the chat-template setting. Output targets of approximately 100, 500 and 1000 tokens are expressed through system instructions. They are not hard generation limits: end-of-sequence behavior and sampling remain active, and actual output lengths are measured. The three targets should therefore be interpreted as workload labels.

The workloads have similar input-token ranges but are not paired translations. In particular, the English 1000-token instruction increases mean input length relative to the other English profiles. Differences between Russian and English results can reflect prompt content, output lengths and sampling as well as language.

TABLE 3.2
Prepared input lengths, including the chat template

Language	Output target	Histories	Min.	Mean	Max.
RU	100	100	70	86.78	120
RU	500	100	71	87.78	121
RU	1000	100	71	87.78	121
EN	100	100	70	85.85	116
EN	500	100	71	86.85	117
EN	1000	100	84	99.85	130

Lengths are tokens. The same 100 histories per language are reused for each output target.

3.4 Load protocol

Every deployment is tested in $2 \times 3 \times 6 = 36$ conditions: two languages, three output targets and six load modes. The first mode maintains one outstanding request (concurrency one). The other five modes use Poisson arrivals at 1, 2, 4, 8 and 16 requested arrivals per second (RPS) [11].

Concurrency one is a closed-loop test: a completion permits the next request. Fixed RPS is open-loop: arrivals continue while earlier work remains unfinished. These conditions must be analyzed separately.

Each run has a 30-second warmup and 120 seconds of main load submission. AIPerf permits up to 30 additional seconds for outstanding work to finish. The resulting metrics are not a strict snapshot at second 120. The runner executes conditions sequentially against one already-started server. A full grid has 90 minutes of nominal warmup and submission time, with additional drain and tooling overhead. The archived baseline series lasted approximately 160 minutes in total.

3.5 Metric definitions

Time to first token (TTFT) measures the client-observed delay before the first content is received. Request latency measures time to the end of the response. These are end-to-end measurements rather than isolated GPU execution times.

The repository calls AIPerf's `inter_token_latency` metric time per output token (TPOT). For request i, its implementation computes [20]

$$\mathrm{TPOT}_i = \frac{L_i - F_i}{N_i - C_i}, \tag{3.1}$$

where L_i is request latency, F_i is TTFT, N_i is output-token count and C_i is the first content chunk's token count. When that count is unavailable or inconsistent, the implementation falls back to one. The benchmark requests per-chunk usage. Reported TPOT percentiles summarize these per-request averages; they do not describe the distribution of every individual inter-chunk gap.

The tables report the median (p50), 95th (p95) and 99th (p99) percentiles. Quantiles cannot generally be combined algebraically: median TPOT multiplied by median output length need not equal median request latency.

Output throughput is taken directly from the exported AIPerf `output_token_throughput` field. Its implementation divides aggregate output-token count by the metric observation duration [21]. That duration should not be replaced with a hard-coded 120 seconds or assumed equal to the separately exported `benchmark_duration`. Cancelled requests and the drain period affect interpretation. Submitted, completed, cancelled and error counts are therefore reported alongside throughput.

For speculative deployments, let A_i, D_i and V_i be accepted draft tokens, proposed draft tokens and verification steps for request i. The planned run-level acceptance metrics are [22]

$$\mathrm{AcceptanceRate} = 100\frac{\sum_i A_i}{\sum_i D_i}, \tag{3.2}$$

$$\mathrm{AcceptanceLength} = 1 + \frac{\sum_i A_i}{\sum_i V_i}. \tag{3.3}$$

The latter includes one bonus token per verification step according to the tool's accounting. These are ratios of aggregated counters rather than averages of per-request percentages. Both are not applicable to a deployment without a drafter.

3.6 Optimization and comparison protocol

The proposed process starts with validation and repeated measurements of B_0. Engine settings are then searched to establish B_1. Speculative and quantized candidates are first tested individually, followed by compatible combinations. Profiling determines whether a kernel or scheduling change is justified. After each stage, the same end-to-end grid is rerun and any regression is retained in the record.

For a latency statistic L and throughput T, comparisons use

$$S_L = \frac{L_{\mathrm{baseline}}}{L_{\mathrm{candidate}}}, \qquad S_T = \frac{T_{\mathrm{candidate}}}{T_{\mathrm{baseline}}}. \tag{3.4}$$

The workload, load mode, metric definition and completion policy must match. A configuration that changes output length or cancels more requests cannot be declared better from its latency ratio alone. Fixed-length diagnostic runs may be

added as a separate experiment; they will not silently replace the instruction-based baseline.

Component-removal experiments will compare the full configuration with versions that remove one optimization at a time. Selected pairwise experiments will help determine whether improvements compose and whether optimization order matters. For RQ3, configuration rules will be selected using development workloads and evaluated on newly reserved histories that were not used for tuning.

[Pending: E1–E6: specify search ranges, compatible combinations and the final workload-selection rule. Run at least three independently seeded repetitions of retained configurations and report run-to-run uncertainty.]

3.7 Quality and validity controls

The archive contains no quality scores. Future experiments must define a separate quality evaluation and acceptable degradation before configuration selection. Exact speculative sampling does not establish downstream quality: quantized targets still require comparison with BF16.

[Pending: Q1: choose quality tasks, language coverage, decoding settings and an acceptable score-change threshold. Define latency and completion targets before making a constrained-throughput claim.]

One run per condition, small low-concurrency samples and first-shard selection limit statistical precision and representativeness. The archive lacks request traces and successful GPU telemetry; it cannot establish queue dynamics, peak memory or operation-level bottlenecks. Repetition, longer load tests and profiling are required.

Chapter 4

Implementation

4.1 Repository organization

The implementation separates prompt preparation, model serving, load generation and result reporting. The reviewed repository snapshot is commit `3bd7f18`; the baseline manifest records experiment commit `db89980` with an additional archived launcher patch [4], [5]. These identifiers describe different stages of the same experiment record and should not be interchanged.

The core files are:

- `scripts/prepare_wildchat_prompts.py`: dataset filtering, chat-template token counting and request-file generation;
- `scripts/serve_baseline.sh` and `scripts/serve_dflash.sh`: target-only and speculative server launchers;
- `scripts/benchmark.sh`: one AIPerf workload invocation;
- `scripts/run_benchmarks.py`: sequential execution of the 36-condition grid and incremental report generation;

- `reports/baseline_20261013_092255/`: the archived manifest, 36 JSON summaries, request counts and formatted tables.

The thesis tables and figures are generated from these archived summaries. A separate script supplied with the manuscript extracts the measurements into a CSV file and rebuilds the presentation without running model inference.

4.2 Prompt preparation and load generation

The preparation stage constructs chat requests using the local Qwen tokenizer. System instructions vary by language and approximate response length. Each resulting JSONL file contains 100 request payloads, with thinking disabled. Input metadata records selection parameters and token-length statistics, allowing the intended workload to be checked before serving.

The benchmark sends streaming requests to the local OpenAI-compatible chat endpoint. Server token counts and per-chunk usage are enabled. The runner changes the prompt file and load mode while keeping one deployment active. Each series uses its own output directory; the runner writes tables as conditions finish so that partial progress remains inspectable [4], [11].

A run-level completion flag only indicates that the benchmark process produced a completed report. It does not establish that every submitted request finished. The baseline demonstrates why the implementation must retain request-level completion counters alongside the metric summaries.

4.3 Baseline launcher and provenance

The target-only launcher loads Qwen3-8B in BF16, disables prefix caching and retains the normal CUDA-graph execution path. Sampling uses the recorded generation defaults: temperature 0.6, top-p 0.95 and top-k 20. The manifest also records model configuration, package versions, launch commands and hashes of the prompt files and scripts [5].

The archived `launch-fix.patch` removes a speculative-metrics option from the no-drafter launcher. This patch is part of the measured configuration. Reproducing only the experiment commit without its patch would therefore omit a recorded change. The input and script checksums in the archive match the corresponding files in the reviewed snapshot.

The manifest is a useful starting point but not a complete environment lock. Future deployments should additionally preserve checkpoint revisions, resolved engine arguments, hardware and driver details, random seeds and the exact benchmark output directory. Raw request traces and server logs should accompany the summary tables.

4.4 Prepared DFlash integration

A DFlash launcher already exists. It references a local copy of the Qwen3-8B DFlash block-size-16 checkpoint and requests 15 speculative tokens [10], [11]. It also enables the speculative-metrics collection needed for acceptance statistics.

The procedure documents a compatibility workaround: disabling FlashInfer autotuning avoids a startup test that exceeds the draft model's supported context in the installed setup. CUDA graphs remain enabled [11]. This setting is specific

to the prepared speculative launcher; its existence does not show that the baseline used the same setting.

A matched target-only control with autotuning disabled is therefore required before attributing a measured difference entirely to DFlash. The experiment should first verify startup, valid responses and populated draft counters, then run the complete workload grid. No such results are present in the reviewed archive.

[Pending: E2: run the matched control and DFlash series; add the final checkpoint revision, server command, draft length, acceptance metrics, quality checks and performance comparison.]

4.5 Remaining integration work

Engine tuning will expose a small, documented configuration search rather than changing several unrecorded defaults at once. Quantized candidates require compatible model formats and kernels, plus a separate quality evaluation. Kernel or scheduler changes will be justified by measured bottlenecks. EAGLE-3, DSpark and an SGLang engine comparison are conditional extensions, subject to compatible checkpoints, implementation support and the available experiment budget.

The final integration must preserve a switch for each retained component. This makes it possible to compare the complete pipeline with versions that remove quantization, speculation or a kernel change. Each experiment will produce the same reporting schema, with additional acceptance and quality fields when applicable. The implementation will retain unsuccessful configurations and regressions as evidence about compatibility and workload dependence.

Chapter 5

Evaluation and Discussion

5.1 Available evidence

This chapter reports the completed target-only baseline and defines the remaining comparisons. The archive contains 36 runs conducted on 13 October 2026, approximately 12:28–15:08 Moscow time [5]. All runs produced complete reports. Across the grid, 22,843 requests were submitted, 22,459 completed and 384 were cancelled; the separate error counter is zero. The overall cancellation fraction is 1.68%, although cancellations are concentrated in particular conditions.

All numerical results below are derived from the archived JSON summaries and request-count CSV. Each condition has one measured run. There are no confidence intervals or optimized-model results in this draft. Full baseline percentiles and completion counts are provided in Appendix A.

5.2 Performance at concurrency one

Table 5.1 summarizes the closed-loop measurements. Median TTFT is between 12.71 and 13.89 ms, and median TPOT is between 6.50 and 6.59 ms. Median response latency increases with actual response length, while five of the six exported throughput measurements are close to 150–151 output tokens/s.

TABLE 5.1
Baseline at concurrency one

Profile	TPOT p50	Latency p50	Tokens/s	Output p50	Done/sent
RU 100	6.50	0.65	150.26	98	169/169
EN 100	6.50	0.68	150.55	103	135/135
RU 500	6.58	3.43	151.22	520	34/34
EN 500	6.57	3.42	41.31	520	12/13
RU 1000	6.59	5.82	151.14	882	22/22
EN 1000	6.59	6.36	151.25	965	19/19

TPOT is in milliseconds; response latency is in seconds. Profile numbers denote requested output targets, not fixed lengths.

The English 500-token condition is an exception: it reports 41.31 tokens/s, with 12 completed requests out of 13 submitted. Its median TPOT remains 6.57 ms. This result must not be interpreted as evidence of a slower decoding kernel. The small sample, cancellation and metric observation window require investigation using raw traces and a repeated run. The archived summaries alone do not establish the cause.

The requested output target also does not bound observed length. In the English 100-token, concurrency-one run, the median is 103 tokens but p99 is 1266.60 tokens. This long tail contributes to the difference between typical and tail response latency. In addition, the 500- and 1000-token concurrency-one runs

contain only 12–34 completed requests each, limiting the interpretation of high percentiles.

[Pending: E0: repeat the baseline, investigate the English 500-token anomaly and retain request traces. Add uncertainty estimates before using these measurements as final speedup denominators.]

5.3 Behavior as offered load increases

Figure 5.1 shows the five open-loop load levels. Aggregate throughput increases across the tested rates for all six profiles. This increase reflects more work completed per unit time and should be considered together with individual-request latency and completion fraction.

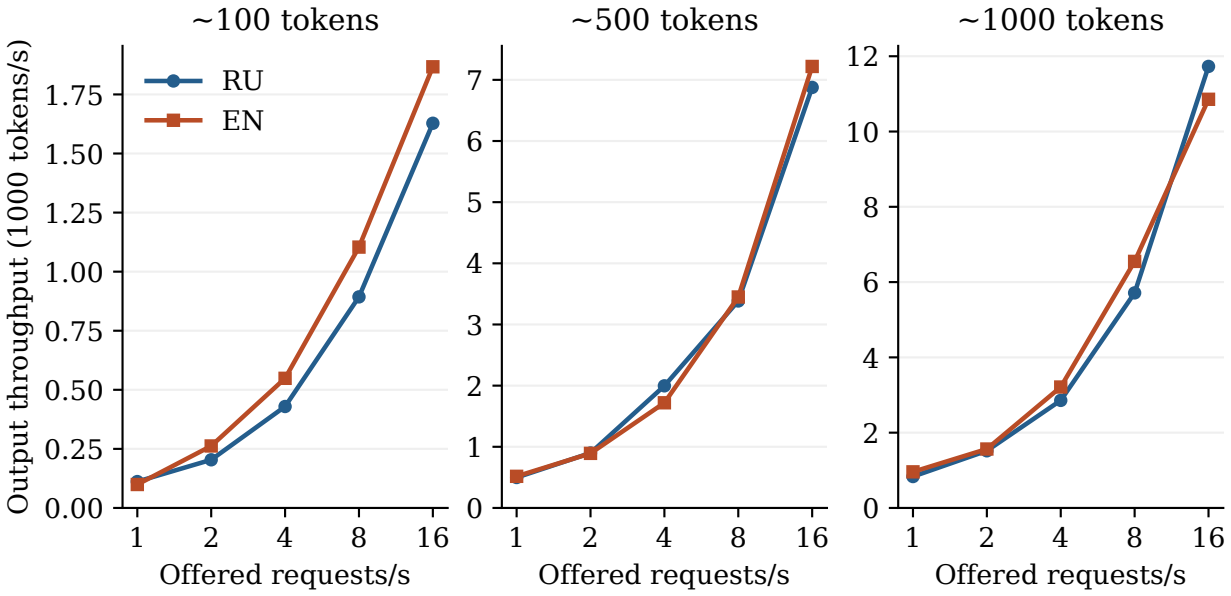


Fig. 5.1. Exported baseline output throughput versus offered RPS. Each point is one run; lines connect measured conditions. Output targets are instructions, not fixed lengths.

At 16 RPS, the 100-token profiles complete all submitted requests and have p99 TTFT below 27 ms. The 500-token profiles reach 6876.25 tokens/s in Russian

and 7219.03 tokens/s in English, with a small number of cancellations. The 1000-token profiles incur much greater response latency, as shown in Table 5.2.

TABLE 5.2
Baseline at 16 offered requests/s

Profile	TTFT p99 (ms)	Latency p99 (s)	Tokens/s	Done/sent
RU 100	25.73	2.54	1628.00	1878/1878
EN 100	26.50	3.78	1866.97	1955/1955
RU 500	39.74	8.70	6876.25	1953/1956
EN 500	41.94	10.98	7219.03	1950/1952
RU 1000	99.37	26.33	11729.77	1936/1938
EN 1000	12180.92	62.72	10856.48	1626/1972

The English 1000-token profile is the clearest high-load concern. Between 8 and 16 RPS, median TPOT rises from 11.19 to 31.23 ms and p99 TTFT rises from 53.20 ms to 12.18 s. At 16 RPS, 346 of 1972 submitted requests are cancelled: only 82.45% complete. The p99 response latency of completed requests is 62.72 s. Figure 5.2 presents this change alongside completion fraction.

These observations are consistent with substantial serving pressure in that condition, but they do not isolate its cause. Queue traces and GPU profiling are needed to distinguish scheduling, memory, compute and host-side effects. The Russian 1000-token condition reaches 11,729.77 tokens/s at 16 RPS with 1936 of 1938 requests completed, but its p99 response latency is still 26.33 s. Different generated lengths and prompt contents prevent treating this comparison as a controlled language effect.

The tested grid does not establish a maximum sustainable request rate. That claim would require longer runs, predefined latency and completion targets, and finer sampling around the capacity boundary. High throughput alone is insufficient when a sizeable share of work remains unfinished.

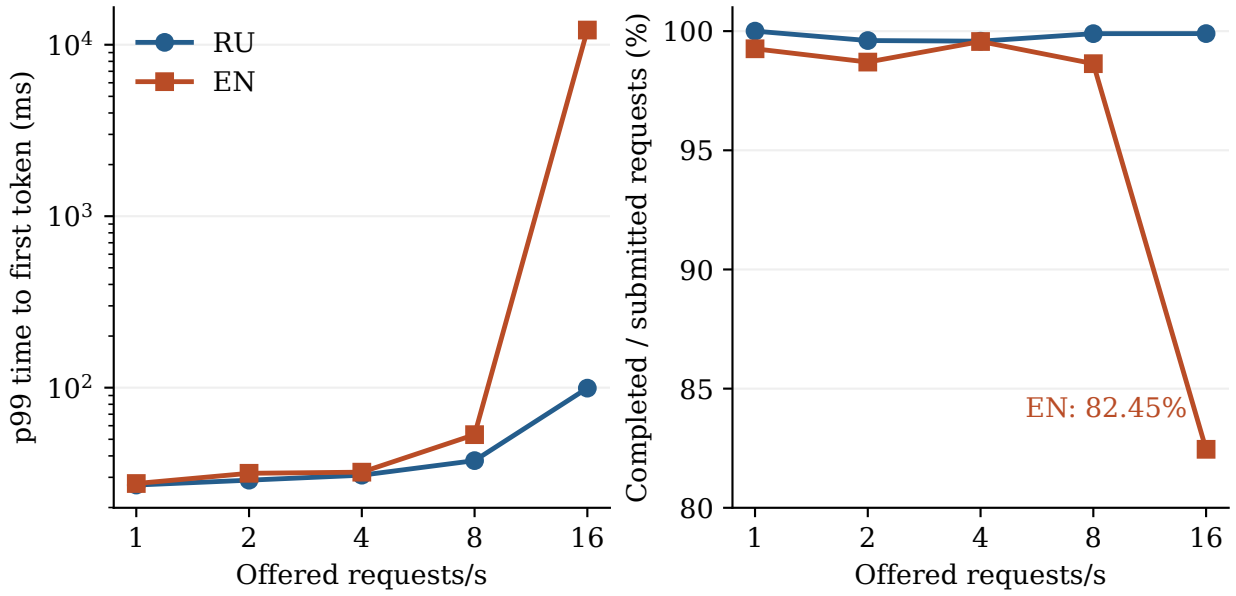


Fig. 5.2. The 1000-token profiles: p99 TTFT for completed requests and completion fraction. The TTFT axis is logarithmic. Cancelled requests are absent from latency percentiles.

5.4 Planned optimization comparisons

The central thesis results will extend the baseline in this chapter. Each retained configuration will receive the complete 36-condition table, including acceptance metrics where applicable. Comparisons will report both typical and tail latency, throughput, output lengths and completion counts. Peak GPU memory and quality scores will be added when measured.

[Pending: E1–E4: insert per-method tables and explain both improvements and regressions. Report measured speedups without assuming that published gains transfer to this setup.]

[Pending: E5: insert the cumulative comparison and component-removal figure. Quantify whether combined gains are additive, smaller than expected or workload dependent.]

TABLE 5.3
Experiments required to complete the evaluation

ID	Comparison	Result to insert
E1	Tuned engine vs. B_0	Configuration search, retained B_1, latency and throughput changes.
E2	DFlash vs. matched control	Draft-cost and acceptance analysis, workload-specific gains or regressions.
E3	Quantization vs. BF16	Precision, calibration, quality change, memory and performance.
E4	Profile-guided changes	Bottleneck evidence, selected kernel or scheduler change, end-to-end effect.
E5	Combined pipeline	Full configuration and component-removal results against B_0 and B_1.
E6	Workload-based selection	Held-out evaluation of configuration rules and their failure cases.

[Pending: E6 and Q1: insert held-out workload and quality results. State the final selection rule only after its validation.]

5.5 Current answers to the research questions

For RQ1, the baseline provides reference measurements; achievable acceleration remains pending until the combined pipeline and matched controls are evaluated. For RQ2, interactions between speculation, precision and execution require individual and component-removal experiments. For RQ3, load and response profile clearly affect baseline behavior, but a configuration-selection rule has not yet been validated.

The baseline shows why throughput, tail latency and completion must be reported together. It establishes no optimized speedup, quality-preservation result or memory reduction. Those claims will depend on the remaining experiments and their uncertainty estimates.

Chapter 6

Conclusion

This thesis investigates a staged optimization process combining serving configuration, speculative decoding, quantization and targeted GPU execution changes. The intended contribution is evidence about which combinations improve end-to-end performance under defined quality and completion constraints.

The completed work establishes bilingual workloads, a benchmark procedure and 36 Qwen3-8B BF16 baseline measurements on one H100. The analysis identifies variable response lengths, an anomalous concurrency-one run and substantial cancellations in one high-load condition. These limitations require validation before final comparisons.

The remaining work is to repeat the baseline, evaluate the prepared DFlash integration with matched controls, and test supported precision and engine configurations. Profiling, component-removal experiments and held-out quality evaluation will support the final conclusions.

[Pending: Final conclusion: insert measured answers to RQ1–RQ3, supported performance gains, quality and completion constraints, and remaining limitations.]

References

[1] W. Kwon et al., *Efficient memory management for large language model serving with PagedAttention*, 2023. arXiv: 2309.06180.

[2] J. Chen, Y. Liang, and Z. Liu, *DFlash: Block diffusion for flash speculative decoding*, 2026. arXiv: 2602.06036.

[3] A. Agrawal et al., *Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve*, 2024. arXiv: 2403.02310.

[4] B. Sharipov. "iu-diploma-2027: Experiment implementation, snapshot 3bd7f18," Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/bulatsharif/iu-diploma-2027/tree/3bd7f18d184cdad8101fff41fdf53125a0fe085f

[5] B. Sharipov. "Qwen3-8B: Baseline without a drafter, 13 october 2026," Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/bulatsharif/iu-diploma-2027/tree/3bd7f18d184cdad8101fff41fdf53125a0fe085f/reports/baseline_20261013_092255

[6] L. Zheng et al., *SGLang: Efficient execution of structured language model programs*, 2024. arXiv: 2312.07104.

[7] Z. Ye et al., *FlashInfer: Efficient and customizable attention engine for LLM inference serving*, 2025. arXiv: 2501.01005.

[8] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202, 2023, pp. 19 274–19 286. [Online]. Available: https://proceedings.mlr.press/v202/leviathan23a.html

[9] Y. Li, F. Wei, C. Zhang, and H. Zhang, *EAGLE-3: Scaling up inference acceleration of large language models via training-time test*, 2025. arXiv: 2503.01840.

[10] Z Lab. "Qwen3-8B-DFlash-b16: Draft model card," Accessed: Oct. 13, 2026. [Online]. Available: https://huggingface.co/z-lab/Qwen3-8B-DFlash-b16

[11] B. Sharipov. "Experiment procedure for Qwen3-8B inference benchmarks," Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/bulatsharif/iu-diploma-2027/blob/3bd7f18d184cdad8101fff41fdf53125a0fe085f/docs/experiment-procedure.md

[12] X. Cheng, X. Yu, C. Shao, et al., *DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation*, 2026. arXiv: 2607.05147.

[13] J. Lin et al., *AWQ: Activation-aware weight quantization for LLM compression and acceleration*, 2024. arXiv: 2306.00978.

[14] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202, 2023, pp. 38 087–38 099. [Online]. Available: https://proceedings.mlr.press/v202/xiao23c.html

[15] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, *FlashAttention-3: Fast and accurate attention with asynchrony and low-precision*, 2024. arXiv: 2407.08608.

[16] T. Zadouri, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao, *FlashAttention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling*, 2026. arXiv: 2603.05451.

[17] Qwen Team. "Qwen3-8B: Model card," Accessed: Oct. 13, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3-8B

[18] A. Yang, A. Li, B. Yang, et al., *Qwen3 technical report*, 2025. arXiv: 2505.09388.

[19] W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi, and Y. Deng, *WildChat: 1m chatgpt interaction logs in the wild*, 2024. arXiv: 2405.01470.

[20] AIPerf Contributors. "AIPerf v0.13.0: Inter-token latency metric implementation," Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/ai-dynamo/aiperf/blob/v0.13.0/src/aiperf/metrics/types/inter_token_latency_metric.py

[21] AIPerf Contributors. "AIPerf v0.13.0: Output-token throughput metric implementation," Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/ai-dynamo/aiperf/blob/v0.13.0/src/aiperf/metrics/types/output_token_throughput_metrics.py

[22] AIPerf Contributors. "AIPerf v0.13.0: Speculative decoding metrics implementation," Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/ai-dynamo/aiperf/blob/v0.13.0/src/aiperf/metrics/types/spec_decode_metrics.py

Appendix A

Complete Baseline Results

The following tables reproduce all 36 conditions from the archived baseline [5]. Quantiles are p50/p95/p99. The tables retain the exported values; they do not recompute throughput from elapsed wall time. Concurrency one (C1) is a closed-loop condition, while the other rows identify offered Poisson RPS.

Acceptance rate and acceptance length are not applicable to every row because the baseline has no drafter. Latency and output-length distributions describe completed requests. Table A.1 lists the corresponding request outcomes; those counts must be consulted when interpreting high-load percentiles. A zero error count does not imply zero cancellations.

TABLE A.1
Baseline request outcomes. Error counts are zero for all runs.

Lang.	Target	Load	Sent	Done	Cancelled	Done (%)
RU	100	C1	169	169	0	100.00
RU	100	RPS 1	123	123	0	100.00
RU	100	RPS 2	234	234	0	100.00
RU	100	RPS 4	492	492	0	100.00
RU	100	RPS 8	1020	1020	0	100.00
RU	100	RPS 16	1878	1878	0	100.00
EN	100	C1	135	135	0	100.00

Table A.1 continued

Lang.	Target	Load	Sent	Done	Cancelled	Done (%)
EN	100	RPS 1	95	95	0	100.00
EN	100	RPS 2	251	251	0	100.00
EN	100	RPS 4	491	491	0	100.00
EN	100	RPS 8	997	997	0	100.00
EN	100	RPS 16	1955	1955	0	100.00
RU	500	C1	34	34	0	100.00
RU	500	RPS 1	113	113	0	100.00
RU	500	RPS 2	213	213	0	100.00
RU	500	RPS 4	474	474	0	100.00
RU	500	RPS 8	969	968	1	99.90
RU	500	RPS 16	1956	1953	3	99.85
EN	500	C1	13	12	1	92.31
EN	500	RPS 1	115	115	0	100.00
EN	500	RPS 2	247	246	1	99.60
EN	500	RPS 4	469	467	2	99.57
EN	500	RPS 8	933	930	3	99.68
EN	500	RPS 16	1952	1950	2	99.90
RU	1000	C1	22	22	0	100.00
RU	1000	RPS 1	115	115	0	100.00
RU	1000	RPS 2	253	252	1	99.60
RU	1000	RPS 4	474	472	2	99.58
RU	1000	RPS 8	945	944	1	99.89
RU	1000	RPS 16	1938	1936	2	99.90
EN	1000	C1	19	19	0	100.00
EN	1000	RPS 1	135	134	1	99.26
EN	1000	RPS 2	231	228	3	98.70
EN	1000	RPS 4	459	457	2	99.56
EN	1000	RPS 8	952	939	13	98.63
EN	1000	RPS 16	1972	1626	346	82.45
Total			22843	22459	384	98.32

TABLE A.2
Complete baseline: approximately 100 requested output tokens

Lang.	Load	TTFT (ms)	TPOT (ms)	Latency (ms)	Tokens/s	Output length
RU	C1	13.89 / 14.21 / 16.15	6.50 / 6.54 / 6.56	645.29 / 1256.91 / 1823.33	150.26	98.00 / 191.00 / 276.48
RU	RPS 1	18.11 / 23.91 / 26.38	6.52 / 6.62 / 6.64	626.13 / 1609.06 / 3079.67	111.61	94.00 / 240.90 / 463.34
RU	RPS 2	19.93 / 24.10 / 26.33	6.56 / 6.63 / 6.65	647.31 / 1042.78 / 1979.85	203.80	97.00 / 157.05 / 296.36
RU	RPS 4	20.79 / 24.22 / 24.85	6.62 / 6.70 / 6.73	645.16 / 1080.36 / 2430.31	429.33	95.00 / 160.90 / 362.87
RU	RPS 8	20.98 / 24.10 / 24.86	6.70 / 6.80 / 6.87	661.64 / 1092.66 / 2565.95	893.22	97.00 / 160.10 / 375.86
RU	RPS 16	21.43 / 24.63 / 25.73	6.83 / 6.96 / 6.98	670.25 / 1095.39 / 2536.76	1628.00	96.00 / 158.15 / 367.99
EN	C1	13.89 / 14.21 / 16.40	6.50 / 6.54 / 6.59	677.20 / 1333.86 / 8375.18	150.55	103.00 / 203.00 / 1266.60
EN	RPS 1	17.55 / 25.01 / 25.75	6.52 / 6.60 / 6.66	677.04 / 1107.43 / 3514.52	99.24	102.00 / 166.90 / 530.56
EN	RPS 2	21.19 / 24.56 / 24.99	6.57 / 6.72 / 6.77	679.59 / 1198.47 / 2422.12	262.29	102.00 / 177.50 / 366.00
EN	RPS 4	21.30 / 24.85 / 25.44	6.70 / 6.84 / 6.88	705.79 / 1241.95 / 4598.95	548.61	103.00 / 184.00 / 676.80
EN	RPS 8	21.48 / 24.71 / 25.51	6.88 / 7.03 / 7.05	718.11 / 1372.98 / 4105.86	1103.92	103.00 / 196.20 / 597.52
EN	RPS 16	22.15 / 25.57 / 26.50	7.07 / 7.27 / 7.34	731.97 / 1292.79 / 3776.04	1866.97	101.00 / 178.90 / 533.36

Triplets are p50 / p95 / p99. C1: concurrency one. Acceptance rate and length: not applicable (no drafter). One run per condition.

TABLE A.3
Complete baseline: approximately 500 requested output tokens

Lang.	Load	TTFT (ms)	TPOT (ms)	Latency (ms)	Tokens/s	Output length
RU	C1	13.11 / 14.05 / 17.27	6.58 / 6.59 / 6.59	3426.41 / 5206.00 / 5448.50	151.22	520.00 / 789.20 / 825.01
RU	RPS 1	21.91 / 24.88 / 25.34	6.72 / 6.80 / 6.83	3468.20 / 5171.94 / 6338.87	497.89	514.00 / 773.60 / 937.08
RU	RPS 2	22.08 / 25.25 / 26.17	6.83 / 7.02 / 7.04	3518.96 / 4930.68 / 5496.36	898.45	511.00 / 720.20 / 806.68
RU	RPS 4	22.58 / 26.21 / 27.16	7.12 / 7.19 / 7.20	3586.54 / 5480.63 / 6349.45	1996.78	503.00 / 768.35 / 882.54
RU	RPS 8	24.11 / 27.84 / 29.10	7.48 / 7.79 / 7.88	3810.10 / 5711.12 / 6349.21	3384.47	507.00 / 758.30 / 846.66
RU	RPS 16	30.61 / 36.45 / 39.74	9.19 / 9.83 / 9.92	4689.04 / 7087.67 / 8703.83	6876.25	506.00 / 769.00 / 956.12
EN	C1	12.71 / 15.94 / 17.25	6.57 / 6.59 / 6.59	3422.65 / 6243.44 / 6341.65	41.31	520.00 / 946.85 / 961.37
EN	RPS 1	21.89 / 25.09 / 26.10	6.70 / 6.82 / 6.85	3735.90 / 5079.59 / 6518.86	517.15	551.00 / 750.90 / 961.52
EN	RPS 2	22.19 / 25.59 / 26.14	6.89 / 7.13 / 7.18	3688.22 / 5676.89 / 7046.46	892.39	530.00 / 820.00 / 1025.15
EN	RPS 4	23.33 / 26.66 / 28.12	7.19 / 7.71 / 7.76	3959.92 / 5883.04 / 7110.23	1719.60	542.00 / 802.70 / 968.74
EN	RPS 8	26.05 / 31.24 / 32.88	8.36 / 9.10 / 9.14	4462.11 / 6812.73 / 8834.83	3449.58	537.00 / 816.55 / 1032.52
EN	RPS 16	31.82 / 39.03 / 41.94	9.46 / 10.61 / 10.75	5156.70 / 7887.50 / 10984.60	7219.03	539.50 / 809.55 / 1098.78

Triplets are p50 / p95 / p99. C1: concurrency one. Acceptance rate and length: not applicable (no drafter). One run per condition.

TABLE A.4
Complete baseline: approximately 1000 requested output tokens

Lang.	Load	TTFT (ms)	TPOT (ms)	Latency (ms)	Tokens/s	Output length
RU	C1	13.37 / 16.08 / 16.53	6.59 / 6.60 / 6.60	5821.02 / 7453.72 / 7952.79	151.14	882.00 / 1128.15 / 1203.31
RU	RPS 1	22.77 / 25.92 / 27.04	6.93 / 7.09 / 7.10	6194.95 / 9082.05 / 9650.48	834.39	889.00 / 1328.00 / 1391.82
RU	RPS 2	24.01 / 27.80 / 28.87	7.48 / 7.88 / 7.90	6672.28 / 9297.94 / 10985.18	1518.24	889.00 / 1216.35 / 1413.10
RU	RPS 4	25.30 / 29.62 / 30.88	8.01 / 8.40 / 8.48	7129.46 / 10274.63 / 11528.47	2857.44	890.50 / 1267.95 / 1462.50
RU	RPS 8	29.67 / 35.12 / 37.54	9.16 / 9.54 / 9.56	8028.37 / 11399.90 / 13457.10	5713.41	887.00 / 1231.70 / 1458.70
RU	RPS 16	61.94 / 87.36 / 99.37	17.18 / 19.83 / 20.16	15227.89 / 21912.91 / 26331.95	11729.77	889.50 / 1253.00 / 1506.30
EN	C1	13.42 / 18.17 / 18.86	6.59 / 6.60 / 6.60	6364.03 / 8491.20 / 9200.18	151.25	965.00 / 1285.80 / 1392.36
EN	RPS 1	22.62 / 26.16 / 27.61	7.15 / 7.33 / 7.40	7569.54 / 10534.61 / 16413.31	959.29	1068.50 / 1471.90 / 2281.29
EN	RPS 2	25.49 / 30.33 / 31.67	7.87 / 9.05 / 9.10	7981.59 / 12476.22 / 16771.74	1565.52	999.00 / 1554.60 / 1931.31
EN	RPS 4	25.48 / 29.84 / 32.15	7.91 / 8.49 / 8.54	8222.82 / 12016.80 / 17998.07	3212.48	1038.00 / 1484.20 / 2277.76
EN	RPS 8	36.80 / 47.45 / 53.20	11.19 / 14.00 / 14.31	11896.31 / 18564.57 / 26341.61	6549.94	1040.00 / 1472.50 / 2102.96
EN	RPS 16	105.13 / 4839.34 / 12180.92	31.23 / 43.14 / 48.81	30204.84 / 52836.37 / 62715.52	10856.48	1011.50 / 1434.00 / 1785.25

Triplets are p50 / p95 / p99. C1: concurrency one. Acceptance rate and length: not applicable (no drafter). One run per condition.

Appendix B

Evidence and Completion Checklist

B.1 Traceability

The reviewed repository snapshot is 3bd7f18d184cdad8101fff41fdf53125a0fe085f. The experiment manifest records db899801ab688ea7c2183785430fa5430bb46a93, supplemented by `launch-fix.patch`. The primary source directory is `reports/baseline_20261013_092255/` [5].

The manuscript source package contains the extracted measurements in `data/baseline.csv` and their provenance in `data/provenance.json`. The script `scripts/build_results.py` checks request counters and generates the numerical tables from the repository archive. Its companion `scripts/plot_results.py` rebuilds the figures from the CSV. This separation allows manuscript updates without rerunning inference.

B.2 Outstanding evidence

The draft uses explicit **[Pending: ...]** markers. They identify missing evidence rather than predicted results. The following checklist should be closed before submission:

1. P1: complete the environment and checkpoint record.
2. E0: repeat and validate the baseline, including the anomalous English 500-token run.
3. Q1: define and execute quality evaluation and establish latency/completion targets.
4. E1: establish a tuned target-only baseline.
5. E2: evaluate DFlash with a matched control and valid acceptance counters.
6. E3: evaluate supported quantized models against BF16.
7. E4: profile the retained configurations and measure justified kernel or scheduler changes.
8. E5: evaluate the complete pipeline and component-removal controls.
9. E6: test the configuration-selection rule on reserved workloads.
10. Replace the provisional conclusion with evidence-based answers; confirm the supervisor and final institutional cover requirements.

The quality suite, exact search space and any additional model or GPU remain to be selected. These choices should be recorded before the corresponding comparisons so that the final thesis clearly separates exploratory tuning from validation.